

PCMO Project Reference Guide

Architecture, feature catalogue, operations, and launch readiness

Project & Contracts Management Organisation

1. Purpose and current state

PCMO is a full-stack web platform for the Project & Contracts Management Organisation. It combines a public marketing website with authenticated student/member and administrator portals.

This guide reflects the repository state reviewed on 18 July 2026. It is intended as the single operational reference for developers, content administrators, and launch planning.

What the platform delivers

- Public website pages for PCMO, membership, resources, certifications, events, networking, contact, FAQs, Terms & Conditions, and Privacy Policy.
- Student accounts with dashboards, memberships, learning, certifications, events, library access, career tools, community profile, and volunteer workflows.
- Admin console for users, courses, learning content, certificates, memberships, billing records, events, library content, pages, community moderation, and reporting.
- MySQL-backed data, JWT authentication, role checks, file uploads, Stripe integration hooks, and Socket.IO realtime refresh.

2. Architecture

```
Browser<br/> |-- React single-page app (Vite, port 8080)<br/> | |-- Public website routes<br/> | |-- Student routes (JWT required)<br/> | -- Admin routes (admin/super_admin role required)<br/> |<br/> -- /api proxy --> Express TypeScript API (port 3001)<br/> |-- JWT authentication and Zod validation<br/> |-- Multer file uploads to public/uploads<br/> |-- Stripe checkout/webhook integration<br/> |-- Socket.IO notifications/realtime events<br/> `-- MySQL 8.4 database (port 3306)
```

Main code locations

Area	Location	Responsibility
Application routes	src/App.tsx	Defines public, student, and admin route protection.
Public site	src/pages/WebsiteHome.tsx, src/pages/*Hub.tsx	Marketing and discovery pages.
Student portal	src/pages/Index.tsx and student page files	Account, learning, events, resources, career, and volunteer flows.
Admin portal	src/pages/Admin*.tsx, src/components/AdminCrudPage.tsx	Data management, reports, moderation, and content tools.
Shared UI	src/components	Navigation, footer, layouts, data tables, rich editor, and access guards.
Client data/API	src/lib/api.ts, src/hooks	API calls, authentication state, and realtime subscriptions.
API	server/index.ts	Express routes, workflows, uploads, Stripe, and Socket.IO.
Data schema	server/schema.sql	MySQL tables and initial data.
Generic resource model	server/resources.ts	Allowed CRUD resources, fields, ownership, and search configuration.
Configuration	.env, .env.example, server/config.ts	Local and production environment variables.

3. Technology stack

Frontend

Technology	Use in PCMO
React 18 + TypeScript	User interface and typed frontend application.
Vite 5	Development server and production build tooling.
React Router 6	Browser routing and protected route structure.
TanStack React Query 5	API data fetching, cache, mutations, and refresh.
Tailwind CSS 3	Responsive styling and design system utilities.
shadcn/Radix UI	Accessible UI primitives including tabs, dialogs, forms, menus, and toasts.
Lucide React	Icons.
React Hook Form + Zod	Form state and validation support.
Recharts	Admin data visualizations.
Socket.IO Client	Realtime refresh and notifications.
jsPDF, html2canvas	Export/print-related front-end capability.

Backend and data

Technology	Use in PCMO
Node.js + TypeScript	API runtime and scripts.

Technology	Use in PCMO
Express 5	REST API routing and middleware.
MySQL 8.4 + mysql2	Persistent platform data.
Docker Compose	Local MySQL service.
JWT + bcryptjs	Seven-day authenticated sessions and password hashing.
Zod	Request payload validation.
Multer	Course, library, community, and profile uploads.
Socket.IO	Realtime server events.
Stripe	Membership and digital product checkout/webhook support.
OpenAI API (optional)	Configured for automatic community chat replies and audit records.

Quality and tooling

Technology	Use in PCMO
ESLint	Static linting configuration.
Vitest + Testing Library	Front-end unit testing framework.
Playwright	Browser/end-to-end test configuration.
tsx	Runs TypeScript server and maintenance scripts directly.
npm	Dependency and script management.

4. Public website features and pages

Public pages use the site header, footer, responsive navigation, PCMO visual language, and a Vite proxy for API requests. Pages under `/pages/:slug` can also be supplied from the `website_pages` MySQL table and managed from the admin portal.

Area	Pages/routes	Main feature
Home and About	<code>/</code> , <code>/pages/about</code>	PCMO landing content, calls to action, about information.
Membership	<code>/pages/membership_and_networking</code> , <code>/pages/membership_packages</code> , membership audience pages, <code>/membership-plans/:slug</code>	Membership overview, plan comparison, plan detail, and checkout entry points.
Certifications	<code>/pages/certifications</code> , <code>/pages/validate_certificate</code>	Certification discovery and public credential validation.
Resources	<code>/pages/resources</code> , <code>/pages/standards</code> , <code>/pages/thought_leadership</code> , <code>/pages/career_resources</code> , <code>/pages/learning</code> , <code>/pages/podcasts</code> , <code>/pages/webinars</code> , <code>/pages/events</code>	Resource hubs, professional guidance, and public event discovery.

Area	Pages/routes	Main feature
Networking	/pages/membership_community, /pages/job_community, /pages/community_chat_rooms, /pages/upcoming_networking_events, /pages/join_the_conversation, /pages/get_involved, /pages/organizations	Community, jobs, chat rooms, networking events, volunteering, and organisational engagement.
Contact	/contact, /pages/contact	Contact form and support route.
Legal	/pages/terms, /pages/privacy	Dedicated Terms & Conditions and Privacy Policy pages. Legal text needs final approval before public launch.
FAQs	/pages/faqs	All 20 FAQs, expandable answers, and category tabs for Membership, Billing & Account, Learning & Certification, and Community & Support.
Dynamic content	/pages/:slug	Database-driven pages with hero, text sections, reference assets, SEO fields, and call to action.

Public-site content operations

- `website_pages` controls editable public pages, page group, menu labels, summaries, hero images, SEO metadata, status, and display order.
- `homepage_sections` controls editable homepage copy, imagery, cards, links, status, and sort order.
- `membership_plans` controls public membership plan data and rich plan-page content.
- Public navigation is structured in `src/lib/publicNavigation.ts`; its links render only when corresponding database pages are published.

5. Student/member portal features

Student pages require an authenticated JWT and are wrapped with the Student route guard.

Page/route	Feature set
/login	Student registration, login, password reset, and administrator sign-in entry.
/dashboard or /student	Aggregated student dashboard with live MySQL data.
/membership	Membership status, selection, and checkout-related actions.
/certifications	View student certifications and credential data.
/certification-quiz	Certification quiz workflow and result capture.
/courses	Browse available courses.
/courses/:id	Course detail, enrolment, materials, module progress, assessments, and quiz access.
/events	Student event list and registrations.
/networking-events	Networking-event filtered view.
/webinars	Member webinar experience.
/library	Protected resource library, downloads, and paid book access.
/ebook/:id	Digital book/resource detail.

Page/route	Feature set
/invoices/:id	Individual invoice view.
/subscriptions	Subscriptions and billing-related membership data.
/career-navigator	Career goals, skills, milestones, and job recommendations.
/account	Profile, login security, communication preferences, privacy controls, payment display, and order history.
/community-profile, /edit-profile	Member profile, professional information, projects, badges, recommendations, and avatar upload.
/community/chat/:slug	Authenticated community chat room with message workflow and optional AI auto-replies.
/community/post/:id	Community discussion posts and comments.
/volunteer, /volunteer/:id	Volunteer opportunity discovery, applications, and hours logging.
/notifications	Authenticated notifications and action links.

Student-facing workflows

1. Register or sign in.
2. Select/activate a membership where eligible.
3. Enrol in learning content and complete course materials.
4. Take course/certification assessments and record attempts/incorrect answers.
5. View certificates and validate public credentials.
6. Register for events, reserve expert rooms, join chats, or participate in community posts.
7. Manage professional profile, career planning data, privacy preferences, and account information.

6. Admin portal features

Admin pages are protected by JWT authentication plus the `admin` or `super_admin` role. The admin sidebar combines fixed operational modules with optional database-defined `admin_modules` entries.

Admin area	Examples of pages/features
Dashboard and reports	/admin, /admin/marketing, /admin/revenue, /admin/analytics; platform metrics, member segmentation, reports, charts, CSV-style exports, recent activity, and audit logs.
User management	/admin/users; account role/status administration. Member projects, badges, recommendations, and connections are managed through generic resource pages.
Education	/admin/education/courses, course workspace paths, /admin/quizzes, /admin/certificates, /admin/enrollment, /admin/reminders, /admin/incorrect-answers; create courses, materials, assessments, questions, enrolments, certificates, reminders, and learning tracking.
Membership and finance	Membership plans, memberships, subscriptions, invoices, event registrations/sales list, balances and financial section reports. Stripe records are stored alongside platform subscription/invoice records.
Events and expert rooms	Create events and webinars, manage expert rooms, reservations, and event registrations.

Admin area	Examples of pages/features
Content management	/admin/library, /admin/content/blog, /admin/content/pages, /admin/content/homepage, /admin/content/additional_pages; manage resources, uploads, blog records, public pages, and homepage content.
Community and moderation	/admin/community-chat, community posts/comments, chat moderation, and AI reply audit records.
Volunteer operations	Volunteer opportunities, applications, and submitted hours.
CRM	/admin/contacts; review contact form submissions.
Configuration	Admin modules, settings-like routes, localization/testimonial placeholders, and logout.

Generic admin data model

server/resources.ts defines the API-managed resources. It includes member profiles/relationships, courses and assessments, certificates, memberships and plans, events, expert rooms, chat rooms/messages/AI replies, subscriptions/invoices, library items/media/book purchases, community posts/comments, quizzes, volunteering, career data, jobs, website pages, homepage sections, and audit/activity records.

All create, update, and delete operations should be treated as privileged actions. The API records audit information for operational traceability.

7. Database and integrations

The MySQL schema contains more than 40 tables. The major domains are:

Domain	Key tables
Identity and member profile	users, profiles, account_preferences, member_connections, member_projects, member_badges, member_recommendations
Membership and finance	memberships, membership_plans, subscriptions, invoices, book_purchases
Learning and certification	courses, course_enrollments, course_materials, course_module_progress, course_assessments, quiz_questions, quiz_attempts, incorrect_answers, certifications
Events and community	events, event_registrations, expert_rooms, expert_room_reservations, community_chat_rooms, community_chat_messages, community_posts, post_comments, ai_chat_replies
Content and CRM	website_pages, homepage_sections, library_contents, library_media, contact_messages, notifications
Volunteer and careers	volunteer_opportunities, volunteer_applications, volunteer_hour_logs, career_goals, skills, career_milestones, job_recommendations
Administration	admin_modules, admin_records, audit_logs, admin_activity_access, member_activity_events

Third-party integrations

- **Stripe:** checkout and webhook endpoints exist. Configure test keys for local development and live keys/webhook verification for production.

- **OpenAI:** optional server-side automatic community replies. Never expose `OPENAI_API_KEY` to the frontend.
- **Docker/MySQL:** Docker Compose starts MySQL 8.4 with a persistent named volume.
- **File storage:** current uploads are stored under `public/uploads`. This is acceptable for local development but should be moved to managed object storage before production.

8. Working with the project in Visual Studio Code

Visual Studio Code is the recommended editor because this is a Node.js, TypeScript, Vite, and Docker project. Full Visual Studio can open the folder, but its JavaScript/TypeScript workflow offers no advantage here.

Prerequisites

- Node.js 20 LTS or newer.
- npm (installed with Node.js).
- Docker Desktop running locally.
- MySQL client tools are optional but useful for database inspection.
- VS Code extensions: **ESLint**, **Tailwind CSS IntelliSense**, **Docker**, and **MySQL** or a database explorer of your choice.

First-time local setup

1. Open the folder `PCMO` - Copy in VS Code.
2. Open VS Code Terminal (`Terminal > New Terminal`).
3. Install packages if `node_modules` is not already present:

```
```powershell
npm install
```
```

4. Create your local environment file from the template:

```
```powershell
Copy-Item .env.example .env
```
```

5. Edit `.env` and set secure local values for `JWT_SECRET`, `MYSQL_*`, `ADMIN_EMAIL`, and `ADMIN_PASSWORD`. Add Stripe/OpenAI values only when testing those integrations.

6. Start MySQL:

```
```powershell
docker compose up -d mysql
```
```

7. Apply the schema and create the first admin account:

```
```powershell
npm run db:migrate
npm run db:create-admin
```
```

```

8. Start both the frontend and API:

```powershell

npm run dev:all

```

9. Open `http://localhost:8080/`. The API health endpoint is `http://127.0.0.1:3001/api/health`.

## Daily development workflow

Run these in separate VS Code terminals if you do not use `dev:all`:

```
npm run dev
npm run dev:api
```

- Edit frontend routes and components in `src`.
- Edit API routes in `server/index.ts`.
- Update schema and seed files in `server` when changing persistent data.
- Vite refreshes the frontend automatically; `tsx watch` restarts the TypeScript API automatically.
- Use the browser network panel and API health endpoint when diagnosing frontend-to-API errors.

## Useful commands

Command	Purpose
<code>npm run dev</code>	Start the Vite frontend on port 8080.
<code>npm run dev:api</code>	Start the Express API on port 3001.
<code>npm run dev:all</code>	Start frontend and API together.
<code>npm run db:migrate</code>	Apply <code>server/schema.sql</code> .
<code>npm run db:create-admin</code>	Create the configured first administrator.
<code>npm run db:sync-pcmo</code>	Run the PCMO world content-sync script. Review its input/output before using in production.
<code>npm run db:seed-course-content</code>	Add course-content seed data.
<code>npm run db:seed-expanded-catalog</code>	Add expanded course/catalogue seed data.
<code>npm run db:seed-navigation-pages</code>	Add navigation page seed data.
<code>npm run typecheck:api</code>	Type-check the API without emitting files.
<code>npm run test</code>	Run Vitest tests.
<code>npm run lint</code>	Run ESLint across the project.
<code>npm run build</code>	Produce the production frontend build in <code>dist</code> .

## VS Code debugging recommendations

- Use two terminal tabs named `WEB` and `API` when running services separately.
- Set breakpoints in `.tsx` files through the browser debugger, and in `server/*.ts` through a Node.js launch configuration if server debugging is needed.
- Never commit `.env`, Stripe live keys, passwords, JWT secrets, or OpenAI keys.

- Check the currently running service output before assuming a frontend issue: the frontend depends on the API at port 3001 through Vite's /api proxy.

## 9. Validation status

The following checks were run on 18 July 2026:

Check	Result	Meaning
npm run build	Passed	Vite produced a production bundle successfully. It reports a bundle-size warning for a main chunk above 500 kB.
npm run typecheck:api	Passed	API TypeScript compiles without type errors.
npm run test	Passed	1 Vitest smoke test passes. This is not sufficient feature coverage for launch.
API health	Passed during local run	/api/health reported MySQL database connectivity.

## 10. Launch readiness assessment

### Estimated readiness: 70%

This is an engineering readiness estimate based on the codebase and local verification, not a security audit or legal approval. The platform has broad feature coverage and working local architecture, but production-critical validation and operating controls are incomplete.

Area	Estimated completion	Reasoning
Public website and navigation	85%	Major public pages, legal pages, FAQs, navigation, and responsive UI exist. Final approved copy, SEO review, accessibility review, and production analytics still need work.
Student portal	75%	Core learning, membership, events, profile, library, career, community, and volunteer flows are represented and API-backed. End-to-end acceptance testing and policy decisions remain.
Admin portal	80%	Extensive CRUD, reports, content, moderation, and education tools exist. Permission review, role separation, UX acceptance, and operating procedures are still required.
Backend and database	75%	TypeScript API, MySQL schema, authentication, validation, audit logs, and health checks exist. Production security, migration discipline, rate limiting, and monitoring are incomplete.
Payments and integrations	55%	Stripe and optional OpenAI paths are implemented/configurable, but live credentials, webhook validation, reconciliation, and failure testing must be completed.
Testing and quality assurance	25%	Build and API typecheck pass, but there is only one automated smoke test and no demonstrated end-to-end suite.

Area	Estimated completion	Reasoning
Deployment and operations	35%	Docker MySQL and local development are documented. No verified CI/CD, production hosting, managed uploads, backups, observability, or disaster recovery process is present.

## 11. Pending tasks before production launch

### P0 — must complete before accepting real users or payments

- [ ] Replace all development/default credentials and secrets with secure production values; rotate any values that were ever shared.
- [ ] Set a strong, unique `JWT_SECRET`; do not rely on the development fallback in `server/config.ts`.
- [ ] Configure production `CLIENT_ORIGIN`, `CLIENT_URL`, API host, HTTPS, CORS allowlist, and secure reverse proxy.
- [ ] Deploy MySQL as a managed service or hardened server with private networking, encrypted backups, restore testing, and least-privilege database users.
- [ ] Move `public/uploads` to managed object storage with private access rules, malware scanning, content-type validation, and lifecycle policies.
- [ ] Complete Stripe production configuration: live keys, webhook endpoint, webhook secret, signature verification test, payment-success/failure/refund flows, invoice reconciliation, and finance-owner sign-off.
- [ ] Obtain legal review and final approval for Terms & Conditions, Privacy Policy, cookies, refunds/cancellations, membership rules, and certification policies.
- [ ] Implement and test account recovery, verification, session expiry, logout, password policy, brute-force protection, and admin-account safeguards.
- [ ] Conduct a security assessment: authorization checks for every admin resource, upload handling, IDOR/access-control testing, input validation, dependency scan, OWASP review, and penetration test.
- [ ] Create monitoring, error logging, uptime checks, alerting, and incident response ownership.
- [ ] Run complete user acceptance testing for public visitor, student, admin, finance, and support roles.

### P1 — strongly recommended for the first production release

- [ ] Add meaningful unit, component, API, and Playwright end-to-end tests for authentication, enrollment, payments, admin CRUD, files, permissions, content publishing, and community moderation.
- [ ] Add CI/CD to run `lint`, tests, typechecks, build, database migration checks, and deployment approvals on every merge.
- [ ] Address the Vite main-bundle warning with route-level lazy loading and review JavaScript performance on mobile networks.
- [ ] Add server-side rate limiting, request-size limits, security headers, CSRF strategy where required, and structured audit/security logs.
- [ ] Add privacy consent/cookie controls if analytics, non-essential cookies, or regional laws require them.
- [ ] Verify all public routes, SEO titles/descriptions, social metadata, sitemap, robots configuration, canonical URLs, and 404 behavior.
- [ ] Define content approval workflows for pages, courses, certificates, library items, chats, and community posts.

- [ ] Define admin operating procedures for refunds, membership changes, course publishing, certificate corrections, removals, and data access requests.
- [ ] Define data retention schedules for account data, financial records, uploads, messages, and audit logs.
- [ ] Verify email delivery/provider setup for password reset, receipts, reminders, notifications, and support messages.

## P2 — post-launch improvements

- [ ] Add product analytics and a consented reporting strategy.
- [ ] Add accessibility audit/remediation against WCAG 2.2 AA.
- [ ] Add localisation strategy if multiple languages/currencies are required; some admin navigation entries currently act as placeholders for future modules.
- [ ] Add support knowledge-base operations, SLA targets, and customer-service tooling.
- [ ] Add load testing for concurrent login, course, event, download, chat, and checkout traffic.
- [ ] Add formal disaster recovery exercise and documented restore time/recovery point objectives.
- [ ] Add a dependency update policy and automated vulnerability alerting.

## 12. Recommended launch sequence

1. Freeze feature scope and approve legal/public copy.
2. Move secrets, database, uploads, API, and frontend to the intended production infrastructure.
3. Configure and test Stripe in a staging environment.
4. Complete P0 security, access-control, backup, and incident-response work.
5. Add P1 automated tests and run end-to-end user acceptance testing against staging.
6. Load final approved pages, membership plans, courses, events, library items, and admin accounts.
7. Perform a soft launch with selected administrators and student testers.
8. Review logs, payment records, support issues, and monitoring alerts.
9. Launch publicly only after each P0 item has an accountable owner and recorded sign-off.

## 13. Known project notes

- The working copy's `.git` directory is not currently recognised by `git status`; restore/repair repository metadata or re-clone from the canonical remote before relying on normal branch, review, or deployment workflows.
- The test suite currently contains one passing example test only. A green test result is not evidence that the full product is launch-ready.
- Local development uses `http://localhost:8080` for Vite and `http://127.0.0.1:3001` for the API. The Vite proxy sends `/api` requests to the API service.
- The frontend build is successful but emits a main JavaScript bundle size warning. This should be addressed before performance-sensitive production launch.
- The public Terms, Privacy, and FAQ pages have been implemented in this workspace; review content ownership and legal approval before publication.

## 14. Suggested ownership

Workstream	Suggested accountable role
Content, pages, courses, events, FAQs	Content administrator / programme lead
Membership pricing, invoices, Stripe reconciliation	Finance owner
Student support and account requests	Customer support lead
Community and AI-reply moderation	Community manager
Security, hosting, backups, monitoring, CI/CD	Technical lead / DevOps owner
Legal pages, privacy, refunds, terms	Legal/compliance reviewer
Acceptance testing and go/no-go approval	Product owner and executive sponsor

For implementation details, start with `README.md`, `package.json`, `src/App.tsx`, `server/index.ts`, `server/resources.ts`, and `server/schema.sql`.